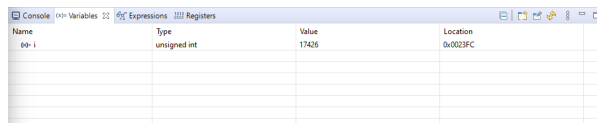


Name: Ryan Ramdihal

Lab 1: *Flashing the LEDs*

Introduction:

In this lab, we delve into utilizing the MSP430 board, using Code Composer Studio (CSS) to create programs that manipulate LED components in different ways. To navigate through this lab we will utilize the User's guide which explains how the MSP430 board works, and the data sheet, with the specifications of the board. Our Lab begins with our initial project on CSS, focusing on flashing the red LED on the MSP430 board. Subsequently, we progress to extending our control to both the red and green LEDs, followed by the adaptation of delay loops to alter flashing speeds. Finally we use what was learned from the previous parts to create a distinctive LED pattern inspired by a Firefighter Truck.



Name	Type	Value	Location
0x-1	unsigned int	17426	0x0023FC

1.2 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0
void main(void){

    volatile unsigned int i;

    WDTCL = WDTPW | WDTHOLD;
    PM5CTL0 &= ~LOCKLPM5;

    P1DIR |= redLED;
    P1OUT &= ~redLED;

    for(;;){
        for(i=0;i<2000;i++){

            P1OUT ^=redLED;

        }
    }
}
```

Flashing the LED (1.2):

The first direct objective is to flash the red LED on the MSP430 board by creating our code in CSS. After naming our project and setting the target to the MSP430F6989. The lab manual includes that the red LED is mapped to Bit 0 or Port 1. By redefining the Bits as masks, it is easier to access in our code. By declaring our loop counter variable 'i' as volatile, the compiler does not try to optimize our delay loop. We disable the timer and clear the lock so we can write new data to memory. We then direct the pin as output and turn the LED off. Inside the while loop, the for loop is used as our delay, where it toggles the LED on.

Flashing two LEDs (1.3):

The second objective is to implement the green LED to the code. By finding out which port and bit the LED is mapped to, from looking through the schematics in the User's guide, we do the same as the red LED and create a mask for it. Setting the port to output and turning the green LED off in the code. After having both LED's blinking in sync, we had to find a way for the LEDs to blink out of sync. To do that we move one of the calls to turn an LED on between the delay loop.

1.3 Schematic:

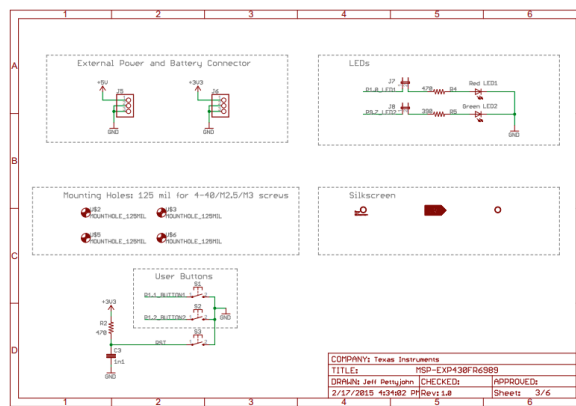


Figure 18. Schematic (3 of 6)

1.3 Blinking in sync Code:

```
#include <msp430fr6989.h>
#define redLED BIT0
#define greenLED BIT7
void main(void){

    volatile unsigned int i;

    WDTCL = WDTPW | WDTHOLD;//disables timer, causes periodical resets
    PM5CTL0 &= ~LOCKLPM5; // clears old data from non-volatile memory

    P1DIR |= redLED; //output
    P1OUT &= ~redLED; //write

    for(;;){
        for(i=0;i<10000;i++){

            P1OUT ^=redLED;
            P9OUT ^=greenLED;

        }
    }
```

1.3 Blinking out of sync Code:

```
#include <msp430fr6989.h>
#define redLED BIT0
#define greenLED BIT7
void main(void){

    volatile unsigned int i;

    WDTCL = WDTPW | WDTHOLD;//disables timer, causes periodical resets
    PM5CTL0 &= ~LOCKLPM5; // clears old data from non-volatile memory

    P1DIR |= redLED; //output
    P1OUT &= ~redLED; //write

    for(;;){

        P1OUT ^=redLED;
        for(i=0;i<2000;i++){

            P9OUT ^=greenLED;
        }
    }
```

Setting a Long Delay (1.4):

In this part, we discover how to create longer delays in between both LEDs. The first way is to have multiple delay loops. Another way is to declare our loop counter variable 'i' as a 32-bit variable to raise its range from the previous 65535 (2^{16}) to (2^{32}). The last solution is to create a delay in CPU cycles. After implementing these solutions, we try creating a delay of about 3 seconds for the flashing.

1.4 2nd Solution Code:

```
#include <msp430fr6989.h>
#include <stdint.h>
#define redLED BIT0
#define greenLED BIT7
void main(void){

    volatile uint32_t i;

    WDTCL = WDTPW | WDTHOLD;//disables timer, causes periodical resets
    PM5CTL0 &= ~LOCKLPM5; // clears old data from non-volatile memory

    P1DIR |= redLED; //output
    P1OUT &= ~redLED; //write

    for(;;){

        P1OUT ^=redLED;
        for(i=0;i<250000;i++){
```

```
    P9OUT ^=greenLED;
}
}
```

1.4 3rd Solution Code:

```
#include <msp430fr6989.h>
#include <stdint.h>
#define redLED BIT0
#define greenLED BIT7
void main(void){

    WDTCTL = WDTPW | WDTHOLD; //disables timer, causes periodical resets
    PM5CTL0 &= ~LOCKLPM5; // clears old data from non-volatile memory

    P1DIR |= redLED; //output
    P1OUT &= ~redLED; //write

    for(;;){

        P1OUT ^=redLED;
        _delay_cycles(3000000);

        P9OUT ^=greenLED;
    }
}
```

Designing Firefighter Truck LED Patterns (1.5):

Our last objective was to create a LED pattern like that of a Firetruck. By implementing what we learned in the previous parts of the lab. I created this pattern by adding 2 for loops for each LED for the number of times the lights should blink and encasing those loops into a while loop, where I can break out the end and move on to a separate while loop with the same pattern but different number of blink iterations. By still having the delay loop in between the two calls for turning the LED on and the other off, the lights still blink out of sync as intended.

1.5 Code:

```
#include <msp430fr6989.h>
#include <stdint.h>
#define redLED BIT0
#define greenLED BIT7
int main(void)
{
    volatile uint32_t i;
    volatile uint32_t j;
    WDTCTL = WDTPW | WDTHOLD; // stop watchdog timer
```

EEL 4742C: Embedded Systems

```
PM5CTL0 &= ~LOCKLPM5;
P1DIR |= redLED;
P9DIR |= greenLED;
P1OUT &= ~redLED;
P9OUT &= ~greenLED;
for(;;){
for(;;){
    for(i=0;i<16;i++){
        P1OUT ^= redLED;
        P9OUT &= ~greenLED;
        delay_cycles(50000);
    }
    delay_cycles(50000);
    for(i=0;i<16;i++){
        P9OUT ^= greenLED;
        P1OUT &= ~redLED;
        delay_cycles(50000);
    }
    break;
}
for(;;)
{
    for(i=0;i<8;i++){
        P1OUT ^= redLED;
        P9OUT &= ~greenLED;
        delay_cycles(50000);
    }
    delay_cycles(50000);
    for(j=0;j<8;j++){
        P9OUT ^= greenLED;
        P1OUT &= ~redLED;
        delay_cycles(50000);
    }
    break;
}
}
}
```

Conclusion:

This lab session provided insightful experience in working with the MSP430 board and Code Composer Studio for manipulating the LED components. With assistance from the User's guide and the data sheet to learn further intricacies of the MSP430's functions. By first focusing on flashing one LED, to both red and green LEDs, creating delays between the speeds of the two, to finally designing a unique LED pattern similar to a Firefighter Truck.

Student Q&A

1. In this lab, we used a delay loop to create a small delay; what is its effect on the battery life if the device is battery operated? Is it a good way of generating delays?

The constant power loss from the delay will drain the battery. Using a delay loop in this format is not efficient in generating delays.

2. The MSP430 CPU runs on a clock that can be slowed down or sped up; what happens to the delay generated by the delay loop if the clock driving the CPU speeds up or slows down?

If the CPU speeds up the delay loop is shortened because it is being processed faster, but if it slows down the processor the clock speed takes longer.

3. How does the code run in the debug mode? Is the microcontroller running as an independent computer?

The debugger runs through the code line by line, with the ability to pause, through each iteration, so it is easier to find and fix any issues/bugs in the code. The microcontroller does not run as an independent computer.

4. How does the code run in the normal mode? Is the microcontroller running as an independent computer?

In normal code the microcontroller runs independently.

5. In which mode does the reset button work?

In normal mode.

6. What is the data type `uint16_t`? What about `int16_t`? Are these standard C syntax?

`uint16_t` represents an unsigned 16-bit integer, `int16_t` is a signed 16-bit integer. These are found in the C library `<stdint.h>` so it is C syntax.